

Mix networks are routing protocols that create hard-to-trace communications by using a chain of proxy servers known as mixes which take in messages from multiple senders, shuffle them, and send them back out in random order to the next destination (possibly another mix node). This breaks the link between the source of the request and the destination, making it harder for eavesdroppers to trace end-to-end communications. Furthermore, mixes only know the node that it immediately received the message from, and the immediate destination to send the shuffled messages to, making the network resistant to malicious mix nodes.

Each message is encrypted to each proxy using public key cryptography; the resulting encryption is layered like a Russian doll (except that each "doll" is of the same size) with the message as the innermost layer. Each proxy server strips off its own layer of encryption to reveal where to send the message next. If all but one of the proxy servers are compromised by the tracer, untraceability can still be achieved against some weaker adversaries.

The concept of a mix "cryptosystem" in the context of electronic mail was first described by David Chaum in 1981 because of the "traffic analysis problem" (traffic analysis). Applications that are based on this concept include anonymous remailers (such as Mixmaster), onion routing, garlic routing, and key-based routing (including Tor, I2P, and Freenet). Large-scale implementations of the mix network concept began to emerge in the 2020s, driven by advancements in privacy-preserving technologies and decentralized infrastructure.

History

David Chaum published the concept of "mixes" in 1979 in a paper for his master's degree thesis work, shortly after he was first introduced to the field of cryptography through the work of public key cryptography, Martin Hellman, Whitfield Diffie and Ralph Merkle. While public key cryptography encrypted the security of information, Chaum believed there to be personal privacy vulnerabilities in the meta data found in communications. Some vulnerabilities that enabled the compromise of personal privacy included time of messages sent and received,

size of messages and the address of the original sender. He cites Martin Hellman and Whitfield's paper "New Directions in Cryptography" (1976) in his work.

1990s: Cypherpunk movement

Innovators like Ian Goldberg and Adam Back made huge contributions to mixnet technology. This era saw significant advancements in cryptographic methods, which were important for the practical implementation of mixnets. Mixnets began to draw attention in academic circles, leading to more research on improving their efficiency and security. However, widespread practical application was still limited, and mixnets stayed largely within experimental stages. A "cypherpunk remailer" software was developed to make it easier for individuals to send anonymous emails using mixnets.

2000s: Inspiration for other anonymous networks

In the 2000s, the increasing concerns about internet privacy highlighted the significance of mix networks (mixnets). This era was marked by the emergence of Tor (The Onion Router) around the mid-2000s. Although Tor was not a straightforward implementation of a mixnet, it drew heavily from David Chaum's foundational ideas, particularly utilizing a form of onion routing akin to mixnet concepts. This period also witnessed the emergence of other systems that incorporated mixnet principles to various extents, all aimed at enhancing secure and anonymous communication.

2010s: Renewed academic interest in mix networks

Entering the 2010s, there was a significant shift towards making mixnets more scalable and efficient. This change was driven by the introduction of new protocols and algorithms, which helped overcome some of the primary challenges that had previously hindered the widespread deployment of mixnets. The relevance of mixnets surged, especially after 2013,

following Edward Snowden's disclosures about extensive global surveillance programs. This period saw a renewed focus on mixnets as vital tools for protecting privacy.

The Loopix architecture, introduced in 2017, integrated several pre-existing privacy-enhancing techniques to form a modern mix network design. Key elements of Loopix included:

"Sphinx" packet format, ensuring unlinkability and layered encryption

Poisson-process-based packet transmission, introducing randomness to prevent traffic correlation attacks.

Exponential mixing delays, making traffic analysis more difficult.

Loop-based cover traffic, where dummy packets (placeholder packets that do not contain actual data) are continuously injected to obscure real data flows.

Stratified mix node topology, optimizing anonymity while maintaining network efficiency.

The rise of blockchain technologies opened new possibilities for scalable decentralized systems, paving the way for large-scale, distributed mix networks.

2020s: First large-scale implementations

Throughout the 2020s, various public and private research and development programs contributed to the realization of the first large-scale mix networks. By 2025, multiple projects—including OKN, HOPR, Katzenpost, Nym (as the overlay for the later NymVPN), and xx.network (led by David Chaum)—are under active development, aiming to enhance privacy-preserving communication on a broader scale.

How it works

Participant A prepares a message for delivery to participant B by appending a random value R to the message, sealing it with the addressee's public key

K

b

$$K_b$$

, appending B's address, and then sealing the result with the mix's public key

K

m

$$K_m$$

M opens it with his private key, now he knows B's address, and he sends

K

b

(
m
e
s
s
a
g
e
,
R
)

$$\{ \displaystyle K_{\{b\}}(message,R) \}$$

to B.

Message format

K

m

(

R

1

,

K

b

(

R

0

,

m

e

s

s

a

g

e

)

,

B

)

→

(

K

b

(

R

0

,

m

e

s

s

a

g

e

)
,
B
)

$$\{ \displaystyle K_{\{m\}}(R1,K_{\{b\}}(R0,message),B) \} \rightarrow (K_{\{b\}}(R0,message),B) \}$$

To accomplish this, the sender takes the mix's public key (

K

m

$$\{ \displaystyle K_{\{m\}} \}$$

), and uses it to encrypt an envelope containing a random string (

R

1

$$R_1$$

), a nested envelope addressed to the recipient, and the email address of the recipient (B). This nested envelope is encrypted with the recipient's public key (

K

b

$$K_b$$

), and contains another random string (R0), along with the body of the message being sent. Upon receipt of the encrypted top-level envelope, the mix uses its secret key to open it. Inside, it finds the address of the recipient (B) and an encrypted message bound for B. The random string (

R

1

$$R_1$$

) is discarded.

R

0

$$R_0$$

is needed in the message in order to prevent an attacker from guessing messages. It is assumed that the attacker can observe all incoming and outgoing messages. If the random string is not used (i.e. only

(

K

b

(

m

e

s

s

a

g

e

)

)

$$(K_{\{b\}}(message))$$

is sent to

B

$$B$$

) and an attacker has a good guess that the message

m

e

s

s

a

g

e

,

$$message'$$

was sent, he can test whether

K

b

(

m

e

s

s

a

g

e

,

)

=

K

b

(

m
e
s
s
a
g
e
)

$$K_b(message') = K_b(message)$$

holds, whereby he can learn the content of the message. By appending the random string

R
0

$$R0$$

the attacker is prevented from performing this kind of attack; even if he should guess the correct message (i.e.

m

e

s

s

a

g

e

,

=

m

e

s

s

a

g

e

$\{\text{displaystyle message}'=\text{message}\}$

is true) he won't learn if he is right since he doesn't know the secret value

R

0

$$R_0$$

. Practically,

R

0

$$R_0$$

functions as a salt.

Return addresses

What is needed now is a way for B to respond to A while still keeping the identity of A secret from B.

A solution is for A to form an untraceable return address

K

m

(

S

1

,

A

)

,

K

x

$$\{ \displaystyle K_{\{m\}}(S1,A),K_{\{x\}} \}$$

where

A

$$A$$

is its own real address,

K

x

$$K_x$$

is a public one-time key chosen for the current occasion only, and

S

1

$$S_1$$

is a key that will also act as a random string for purposes of sealing. Then, A can send this return address to B as part of a message sent by the techniques already described.

B sends

K

m

(

S

1

,

A

)

,

K

x

(
S
0
,
r
e
s
p
o
n
s
e
)

$$K_m(S_1,A),K_x(S_0,response)$$

to M, and M transforms it to

A
,

S

1

(

K

x

(

S

0

,

r

e

s

p

o

n

s

e

)

)

$$A,S1(K_{\{x\}}(S0,response))\}$$

.

This mix uses the string of bits

S
1

$$\{ \displaystyle S1 \}$$

that it finds after decrypting the address part

K

m

(
S
1
,

A

)

$$\{ \displaystyle K_{\{m\}}(S1,A) \}$$

as a key to re-encrypt the message part

K

x

(

S

0

,

r

e

s

p

o

n

s

e

)

$$\{ \displaystyle K_x(S_0, \text{response}) \}$$

. Only the addressee, A, can decrypt the resulting output because A created both

S

1

$$\{ \displaystyle S_1 \}$$

and

K

x

$$K_x$$

.

The additional key

K

x

$$K_x$$

assures that the mix cannot see the content of the reply-message.

The following indicates how B uses this untraceable return address to form a response to A, via a new kind of mix:

The message from A

→

$\{\displaystyle \longrightarrow\}$

B:

K

m

(

R

1

,

K

b

(
R
0
,
m
e
s
s
a
g
e
,

K

m

(
S
1
,
A
)
,

K

x

)

,

B

)

→

K

b

(

R

0

,

m

e

s

s

a

g

e

,

K

m

(

S

1

,

A

)

,

K

x

)

$$\{ \displaystyle K_{\{m\}}(R1,K_{\{b\}}(R0,message,K_{\{m\}}(S1,A),K_{\{x\}}),B)\longrightarrow$$

$$K_{\{b\}}(R0,message,K_{\{m\}}(S1,A),K_{\{x\}}))$$

Reply message from B

→

$$\{\displaystyle \longrightarrow \}$$

A:

K

m

(

S

1

,

A

)

,

K

x

(

S

0

,

r

e

s

p

o

n

s

e

)

→

A

,

S

1

(

K

x

(

S

0

,

r

e

s

p

o

n

s

e

)

)

$$\{ \displaystyle K_{\{m\}}(S1,A),K_{\{x\}}(S0,response) \} \longmapsto A,S1(K_{\{x\}}(S0,response)) \}$$

Where:

K

b

$$\{ \displaystyle K_{\{b\}} \}$$

= B's public key,

K

m

$$K_m$$

= the mix's public key.

A destination can reply to a source without sacrificing source anonymity. The reply message shares all of the performance and security benefits with the anonymous messages from source to destination.

Vulnerabilities

Although mix networks provide security even if an adversary is able to view the entire path, mixing is not absolutely perfect. Adversaries can provide long term correlation attacks and track the sender and receiver of the packets.

Threat model

An adversary can perform a passive attack by monitoring the traffic to and from the mix network. Analyzing the arrival times between multiple packets can reveal information. Since no changes are actively made to the packets, an attack like this is hard to detect. In a worst case of an attack, we assume that all the links of the network are observable by the adversary and the strategies and infrastructure of the mix network are known.

A packet on an input link cannot be correlated to a packet on the output link based on information about the time the packet was received, the size of the packet, or the content of the packet. Packet correlation based on packet timing is prevented by batching and correlation based on content and packet size is prevented by encryption and packet padding, respectively.

Inter-packet intervals, that is, the time difference between observation of two consecutive

packets on two network links, is used to infer if the links carry the same connection. The encryption and padding does not affect the inter-packet interval related to the same IP flow. Sequences of inter-packet interval vary greatly between connections, for example in web browsing, the traffic occurs in bursts. This fact can be used to identify a connection.

Active attack

Active attacks can be performed by injecting bursts of packets that contain unique timing signatures into the targeted flow. The attacker can perform attacks to attempt to identify these packets on other network links. The attacker might not be able to create new packets due to the required knowledge of symmetric keys on all the subsequent mixes. Replay packets cannot be used either as they are easily preventable through hashing and caching.

Artificial gap

Large gaps can be created in the target flow, if the attacker drops large volumes of consecutive packets in the flow. For example, a simulation is run sending 3000 packets to the target flow, where the attacker drops the packets 1 second after the start of the flow. As the number of consecutive packets dropped increases, the effectiveness of defensive dropping decreases significantly. Introducing a large gap will almost always create a recognizable feature.

Artificial bursts

The attacker can create artificial bursts. This is done by creating a signature from artificial packets by holding them on a link for a certain period of time and then releasing them all at once. Defensive dropping provides no defense in this scenario and the attacker can identify the target flow. There are other defense measures that can be taken to prevent this attack. One such solution can be adaptive padding algorithms. The more the packets are delayed, the

easier it is to identify the behavior and thus better defense can be observed.

Other time analysis attacks

An attacker may also look into other timing attacks other than inter-packet intervals. The attacker can actively modify packet streams to observe the changes caused in the network's behavior. Packets can be corrupted to force re-transmission of TCP packets, which the behavior is easily observable to reveal information.

Sleeper attack

Assuming an adversary can see messages being sent and received into threshold mixes but they can't see the internal working of these mixes or what is sent by the same. If the adversary has left their own messages in respective mixes and they receive one of the two, they are able to determine the message sent and the corresponding sender. The adversary has to place their messages (active component) in the mix at any given time and the messages must remain there prior to a message being sent. This is not typically an active attack. Weaker adversaries can use this attack in combination with other attacks to cause more issues.

Mix networks derive security by changing order of messages they receive to avoid creating significant relation between the incoming and outgoing messages. Mixes create interference between messages. The interference puts bounds on the rate of information leak to an observer of the mix. In a mix of size n , an adversary observing input to and output from the mix has an uncertainty of order n in determining a match. A sleeper attack can take advantage of this. In a layered network of threshold mixes with a sleeper in each mix, there is a layer receiving inputs from senders and a second layer of mixes that forward messages to the final destination. From this, the attacker can learn the received message could not have come from the sender into any layer 1 mix that did not fire. There is a higher probability of matching the sent and received messages with these sleepers thus communication is not completely

anonymous. Mixes may also be purely timed: they randomize the order of messages received in a particular interval and attach some of them with the mixes, forwarding them at the end of the interval despite what has been received in that interval. Messages that are available for mixing will interfere, but if no messages are available, there is no interference with received messages.

== References ==